

signus 실데이터 CLI 검증 절차서

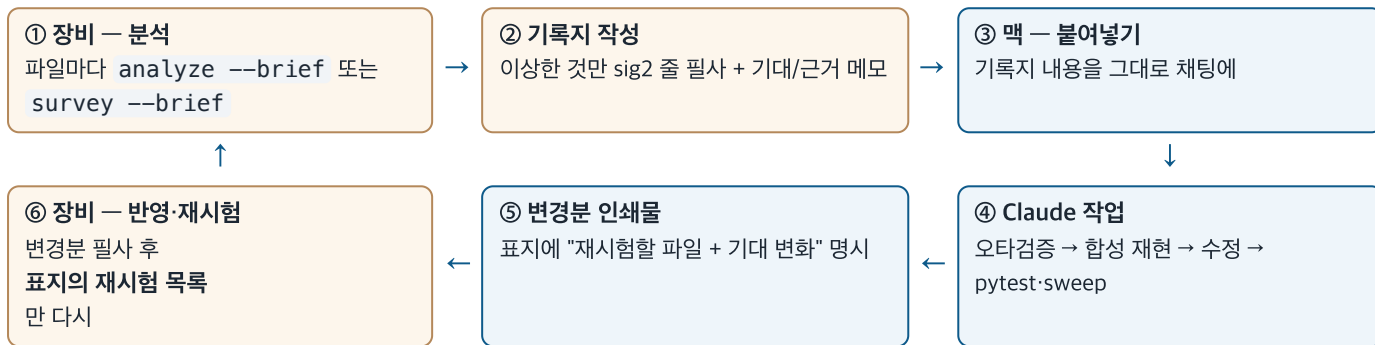
웹 UI 이전 단계 — 격리망 장비에서 무엇을 치고, 무엇을 적어오는지

발급 2026-08-06 · 이 절차서의 명령·출력 예시는 전부 개발기에서 실측한 것 · 기록지는 부록 A·B (필요한 만큼 복사) · 빠른 참조 카드는 부록 C

이 종이의 목적. 장비에서 CLI(`analyze / survey`)로 실패처를 돌려 보고, **이상한 결과만 최소 분량으로** 옮겨 와 Claude 와 코드를 고치는 반복 루프를 돌린다. 설계 최우선 원칙은 **한 번의 방문에서 필요한 것을 다 얻어 오는 것**(두 번 일하지 않기)이다. 잘 나온 결과는 체크 표시만 하고 옮기지 않는다.

전체 그림 — 한 바퀴가 이렇게 돈다

배경색: ■ 격리망 장비(손 필사) ■ 개발기(맥, Claude 가 있는 쪽)



⑥ 이 끝나면 다시 ② 로 — 재시험 결과와 신규 시험을 한 방문에 묶는다.

타키타카 3원칙 (양쪽 다 지킨다)

- 사용자는 본 그대로 옮긴다. 이상해 보여도 값을 고치거나 "보정"하지 않는다 — 오타는 검출 코드가 잡아 주고, 잘못 고친 값은 못 잡는다.
- Claude 는 받은 줄을 무조건 `brief.py check` 부터 돌리고, 재현 없이 추측으로 고치지 않는다. 불일치가 나면 고치기 전에 되묻는다.
- 코드가 장비로 가는 길은 변경분 인쇄물뿐. 재시험 대상과 기대 변화는 변경분 표지에 적혀서 온다 — 그 목록만 다시 돌리면 된다.

0 장비 가기 전 — 맥에서 5분

- ☐ 장비 코드가 어느 인쇄본까지 반영됐는지 기록지 머리에 적는다 (예: "코드북 8/1판 + 변경분 8/5까지"). 결과 줄에는 리비전이 없어서(필사 분량 절약을 위해 뺀 결정) 돌아오면 Claude 가 반드시 물어본다 — 미리 적으면 그 문답이 사라진다.
- ☐ 밀린 변경분이 있으면 필사부터 계획한다. 검증은 항상 "인쇄물과 같은 코드"에서 해야 결과를 해석할 수 있다.
- ☐ 이 절차서 + 기록지(부록 A·B)를 인쇄한다. 기록지는 파일 수만큼.
- ☐ 시험할 캡처를 2단계 매트릭스에서 유형별로 골라 둔다. 첫 방문은 5~10개 권장 — 그 이상은 기록 오타율이 올라간다.

1 장비 도착 — 제일 먼저 자가 점검 (1분)

필사해 둔 저장소 폴더(안에 `signus/` 가 있는 곳)에서:

```
python3 -c "from signus.cli import check_code; print(check_code('sig2 x an fs1 qpsk'))"
```

나온 값	뜻과 행동
<code>psmx</code>	정상. 오늘 옮길 모든 줄의 <code>#검출코드</code> 를 믿을 수 있다. 그대로 진행.
다른 네 글자	<code>cli.py</code> 의 <code>check_code / _ALPHA</code> 부근 필사 오타. 분석은 그대로 진행하되 기록지 머리 "자가점검" 칸에 나온 값을 적는다. 그날 모든 줄이 불일치로 떠도 원인은 값이 아니라 코드 쪽이다 — Claude 가 해당 줄만 다시 인쇄해 준다.
<code>ImportError / SyntaxError</code>	그 파일 필사에 문법 오타. 예러 마지막 줄 (파일명·줄번호 포함)을 적어온다.

명령 표기 약속. 이 절차서의 `signus ...` 는 장비에 명령이 등록돼 있지 않으면 전부 `python3 -m signus.cli ...` 로 바꿔 읽는다. 이하 예시는 짧은 쪽으로 쓴다.

2 무엇을 시험하나 — 유형 매트릭스

"다양한 신호"를 계통적으로: **유형마다 최소 1개**. 오른쪽 끝이 "정상의 모습"이다 — 이 모습이면 ✓ 만 하고 넘어간다.

#	유형	명령	확인하려는 것	정상의 모습
1	제원을 아는 확실한 신호 (규격서 있는 것)	<code>analyze</code>	파이프라인 기준선	변조·baud 일치, <code>lock ≥ 60</code>
2	협대역 연속 디지털	<code>analyze</code>	블라인드 탐지 전반	<code>lock ≥ 60</code> , 제원이 그럴듯
3	버스트 / 패킷성	<code>analyze (+ ---burst N)</code>	버스트 분리, 짧은 패킷	"버스트 N개 감지" 줄, 버스트별 복조
4	FSK 계열	<code>analyze</code>	정포락선 게이트	<code>2fsk/4fsk/msk + h</code> 값
5	광대역 (한 파일에 여러 신호)	<code>survey</code>	탐지 누락·분리	육안으로 보이는 신호 수 = <code>n</code>
6	아날로그 FM 음성 · 톤	<code>analyze/survey</code>	거부가 정상인지	"아날로그/톤" 분류 — 정상도 강제 없음은 설계다
7	약한 신호	<code>analyze</code>	정직한 저품질 보고	<code>lock</code> 낮게 나오되 크래시 없음

순서는 1번부터. 기준선(제원 아는 신호)마저 이상하면 DSP 가 아니라 읽기 설정(`fs/dtype`)이나 필사 쪽 문제일 공산이 크고, 그날 나머지 결과의 해석이 전부 달라진다. 기준선 1~2개로 신뢰를 확보한 뒤 다양한 유형으로 넓힌다.

3 파일 여는 법 — 읽기 필수 정보

`fs`(샘플레이트)와 `iq/real` 여부는 물리적으로 추정 불가능 — 반드시 밖에서 준다. 정답(변조·`fc`·`baud`)은 주는 게 아니다. 사이드카에 있어도 분석기는 절대 안 읽는다(블라인드).

- 파일명에 토권이 있으면 그냥 연다: `<이름>.<cplx|real>.<fs정수>.<샘플타입>[.be][.bitrev][.rf<Hz>].pcm` 예: `capture.cplx.2000000.16t.pcm`
- 임의 이름이면 플래그로: `signus analyze 캡처.bin --fs 2000000 --fmt iq --dtype i16 --brief`
- `ValueError: need fs + fmt` 가 뜨면 이 정보가 빠진 것이다 — 파일명을 고치거나 플래그를 준다.
- 캡처의 RF 중심을 알면 `--rf <Hz>` — "실제 RF" 줄이 추가되고 `survey` 의 중심주파수가 절대값이 된다.

파일명 표기	16t	8t	8o	16o	32f	64f
<code>= --dtype</code>	i16	i8	u8	u16	f32	f64

4 파일마다 이 순서로 — 핵심 절차

4-1. 돌린다

```
signus analyze <파일> --brief      # 단일 신호
signus survey <파일> --brief      # 광대역 (여러 신호가 한 파일에)
```

사람용 출력이 그대로 나오고, 맨 끝에 옮기기용 한 줄이 추가된다:

```
sig2 an fs1000000 iq-i16 16qam fc8000 bd100000 lk100 mer29.8 rl0.33 #vsej
```

4-2. lock 을 보고 셋 중 하나로 판정한다

화면의 lock	판정과 행동
lock ≥ 60, 기대와 일치	믿을 만한 복조. 기록지에 ✓ 와 "변조 lk_" 만 적는다. sig2 줄 필사 불필요 — 잘 나온 결과는 안 가져온다.
기대와 다름 (변조·fc·baud 어느 쪽이든)	마지막 sig2 줄 전체를 필사하고 기대/근거를 메모한다. lock 이 높아도 필사한다 — "자신 있는 오답"이 제일 중요한 표본이다.
lock 0~30 (특히 MER 음수, 롤오프 1.00)	심중팔구 샘플타입이 틀렸다. 그 자리에서 스캔 ↓

```
signus analyze <파일> --dtype i16 --brief      # 넷을 차례로: i16 → f32 → i8 → u8
signus analyze <파일> --dtype f32 --brief      # 필요하면 각 조합에 --be 도 붙여 본다
```

lock 이 확 뛰는 조합이 나오면 그 조합의 sig2 줄을 필사하고 메모 칸에 "스캔: i16→3, f32→88" 식으로 요약. 안 뛰면 "전 dtype 낮음" 한마디면 된다.

4-3. 조건부 표시를 체크박스로 옮긴다

사람용 출력에 뜰 때만 나오는 줄들 — 기록지의 해당 칸에 체크. (sig2 줄에도 실리지만 체크는 필사 검증용 이중화다.)

표시	뜻
버스트 N개 감지	N 을 기록. 기본은 최강 버스트 분석 — 다른 것도 보려면 --burst 2 (1부터)
eq	등화기 적용 (다중경로 보정)
al / fb	반송파 앨리어스 보정 / 심볼레이트 폴백
amb	반송파 앨리어싱 가능 경고 — fc 를 그대로 믿지 말 것 (비트는 대개 멀쩡)
pre	반복 프리앰블 동기화 복조됨

4-4. 필사 요령 (sig2 줄)

- 마지막 줄만, # 뒤 네 글자까지. 공백 위치를 그대로 (공백이 밀리면 값이 달라지는데, 그것도 검출 코드가 잡는다).
- survey 는 신호마다 한 줄 + 맨 끝 #코드 한 번. 13개 이상이면 "...N개 생략" 줄까지 그대로.
- 검출 코드 알파벳에는 o·i·l 이 없다 — o 처럼 보이면 0, i·l 처럼 보이면 1 이다.
- 크래시가 나면 예외의 마지막 줄만 적는다.

5 증상이 보이면 그 자리에서 하나만 더 — 두 번 안 가기

아래 증상이 그 자리에서 보이면, 돌아왔다 다시 가는 일이 없도록 오른쪽 한 가지를 같이 적어온다. 전부 "치는 양"이 한 줄 이하가 되게 골라져 있다.

증상	그 자리에서 하는 일	적어오는 것
lock 은 높는데 변조가 기대와 다름	브라우저가 있으면 <code>signus serve</code> → <code>http://127.0.0.1:8000</code> 에 그 파일을 열어 정상도를 본다	점 덩어리 수 한마디 (예: "4덩어리 링1")
심볼레이트가 배수로 의심 (×2, ×½ 등)	<code>signus analyze <파일> --report r.json</code> 후 <code>python3 -c "import json; print(json.load(open('r.json')) ['detected']['baud_conf'])"</code>	baud_conf 숫자 하나
짧은 패킷이 안 잡힘	아는 대로	버스트 대략 길이(샘플) · 반복 주기 — 숫자 둘
survey 가 신호를 놓침	아는 대로 (워터폴·계획표)	대략 주파수·대역폭 한마디 (예: "+2.3M 근처 200k")
크래시	—	예외 마지막 줄

사진이 가능하면 이 표는 잇는다. 화면 사진 한 장이 위의 어떤 것보다 낫다 — 찍을 수 있는 환경이면 사진으로.

6 돌아와서 — Claude 에게 이렇게 준다

기록지를 이 형식으로 채팅에 옮긴다 (머리 두 줄 + 파일별 블록):

```
자가점검: psmx
장비 코드: 코드북 8/1판 + 변경분 8/5까지

[1] beacon433 — 기대: qpsk 32k 부근 / 근거: 규격서
sig2 an fs2000000 iq-i16 qpsk fc11987 bd32001 lk91 mer24.0 rl0.35 #k2vw
메모: 버스트 3개 감지, 최강만 분석

[2] widecap — 기대: 신호 5개 / 근거: 워터폴 육안
sig2 sv fs2000000 iq-i16 n4
0 fc-6200000 bd200000 lk93 qpsk
1 fc-1500000 bd400000 lk100 16qam
2 fc3099983 bd150000 lk100 msk
3 fc7400000 bd300000 lk93 8psk #7ckr
메모: 누락 의심 +2.3M 근처 폭 200k

[3] voicecap — 기대: FM 음성 / 근거: 육안
✓ 아날로그 분류 (설계대로 거부)
```

✓ 만 한 파일도 위 [3] 처럼 한 줄로 존재만 알려 주면 유형 커버리지를 같이 확인할 수 있다.

그다음 일어나는 일 — Claude 쪽 약속

- 받은 줄마다 즉시 `tools/brief.py check`. 불일치면 고치지 않고 되묻는다 — 그때는 종이 원본을 다시 봐 달라. (본문 오타인지 코드 네 글자 오타인지도 갈라서 알려준다.)
- 아는 제원으로 합성 재현을 시도하고, 재현되면 `tests/` 에 잠그고 고친다 → `pytest + sweep --tier core` 로 정상 신호가 안 깨졌는지 확인.
- 재현이 안 되면 그 사실을 먼저 말하고, 5단계 표에서 딱 한 가지만 더 요청한다.
- 수정이 생기면 변경분 인쇄물을 발급하고, 표지에 재시험 목록 ("beacon433 — 기대: lock 30 → 60")을 명시한다. 다음 방문 때 그 목록과 신규 시험을 묶는다.

7 웹 UI 단계로 넘어가는 기준

2단계 매트릭스의 7유형이 전부 "기대대로" 또는 "문서화된 한계로 설명됨" (예: $\pi/4$ -DQPSK 는 qpsk 표시가 정상, 아날로그는 거부가 정상)이 되면 CLI 검증을 종료한다. 그다음 같은 파일들을 웹 UI(`signus serve` — 애니메이션 성상도·워터폴·일괄 분석)로 육안 확인하는 단계로 넘어간다.

부록 A — analyze 기록지 (파일당 한 칸 · 필요한 만큼 복사)

자가점검(1단계): 장비 코드: 코드북 + 변경분 까지 날짜:

파일: 크기(같은 이름 재시험 때만):

기대: 근거: ☐ 규격서 ☐ 타도구 ☐ 육안 ☐ 없음(블라인드)

플래그: --fs --fmt --dtype ☐ be ☐ rf

판정: ☐ 기대 일치 (변조 lk) — 여기서 끝 ☐ 이상 → 아래 필사 ☐ 크래시 → 마지막 줄을 아래에

조건부: 버스트 N= ☐ eq ☐ al ☐ fb ☐ amb ☐ pre

↑ sig2 줄 필사 (한 칸에 한 글자 · 공백도 한 칸 · # 뒤 네 글자까지)

스캔/메모:

파일: 크기(같은 이름 재시험 때만):

기대: 근거: ☐ 규격서 ☐ 타도구 ☐ 육안 ☐ 없음(블라인드)

플래그: --fs --fmt --dtype ☐ be ☐ rf

판정: ☐ 기대 일치 (변조 lk) — 여기서 끝 ☐ 이상 → 아래 필사 ☐ 크래시 → 마지막 줄을 아래에

조건부: 버스트 N= ☐ eq ☐ al ☐ fb ☐ amb ☐ pre

↑ sig2 줄 필사 (한 칸에 한 글자 · 공백도 한 칸 · # 뒤 네 글자까지)

스캔/메모:

파일: 크기(같은 이름 재시험 때만):

기대: 근거: ☐ 규격서 ☐ 타도구 ☐ 육안 ☐ 없음(블라인드)

플래그: --fs --fmt --dtype ☐ be ☐ rf

판정: ☐ 기대 일치 (변조 lk) — 여기서 끝 ☐ 이상 → 아래 필사 ☐ 크래시 → 마지막 줄을 아래에

조건부: 버스트 N= ☐ eq ☐ al ☐ fb ☐ amb ☐ pre

↑ sig2 줄 필사 (한 칸에 한 글자 · 공백도 한 칸 · # 뒤 네 글자까지)

스캔/메모:

부록 B — survey 기록지 (광대역 파일당 한 칸)

파일: _____ 기대 신호 수: _____ 근거: ☐ 워터폴 육안 ☐ 계획표 ☐ 없음

플래그: --fs _____ --fmt _____ --dtype _____ --rf _____ 나온 n = _____

판정: ☐ 수·제원 모두 기대대로 (✓ 만 하고 끝) ☐ 이상 → 아래에 전 블록 필사 (신호마다 한 줄, 맨 끝 #코드)

[illegible]

-- ✂ 여기부터 잘라 장비 옆에

부록 C — 빠른 참조 카드

sig2 한 줄 읽는 법

```
sig2 an fs1000000 iq-i16 16gam fc8000 bd100000 lk100 mer29.8 rl0.33 #vsej
```

토막	뜻
an / sv	analyze / survey
iq-i16	포맷-샘플타입 — lock 0 의 1순위 용의자라 항상 실린다
fs fc bd	샘플레이트 · 반송파 · 심볼레이트 (Hz 정수)
lk mer rl / h	lock(0~100) · MER(dB) · 롤오프 (FSK 는 변조지수 h)
b2/5	버스트 5개 중 2번째 (하나면 안 나옴)
eq al fb amb pre	등화기 · 앨리어스보정 · baud풀백 · 앨리어싱경고 · 프리앰블 (해당 시만)
#vsej	오타 검출 코드 — 알파벳에 o·i·l 없음 (o→0, i·l→1 로 읽기)

판단 기준 세 줄

- lock ≥ 60 복조 신뢰 · lock 0~30 + MER 음수 + 롤오프 1.00 = 샘플타입 오독 시그니처 → dtype 스캔.
- fc 만 이상 + amb → fc 는 원리상 fs/sym 모듈로만 결정 — 비트는 대개 멀쩡.
- $\pi/4$ -DQPSK 가 qpsk 로 나오는 것, 아날로그·톤·처프의 성상도 거부는 설계된 동작.

자가점검 정상값

```
python3 -c "from signus.cli import check_code; print(check_code('sig2 x an fs1 qpsk'))" → psmx
```